UNIT - 1

1. Define string and language in automata theory.
Answer:
In automata theory, a **string** is a finite sequence of symbols selected from a given alphabet Σ. For example, if Σ = {a, b}, then "abba" is a string. A **language** is a set of strings formed over an alphabet. Languages may be finite or infinite and are used to describe patterns recognized by automata.

2. What is the difference between Σ* and Σ⁺?
Answer:
Σ^* represents the set of all possible strings over alphabet Σ, **including the empty string ε**. Σ^+ represents all possible strings over Σ **excluding ε**. Thus, Σ^* allows zero or more symbols, while Σ^+ allows one or more symbols only.

3. Give one real-life example of FSM application.
Answer: A **traffic light control system** is a real-life application of FSM. Each signal color represents a state. Transitions occur based on timer events. FSM ensures orderly switching between red, yellow, and green lights, maintaining traffic discipline and safety.

4. What is a regular language?
Answer: A **regular language** is a language that can be accepted by a finite automaton such as a DFA or NFA. It can also be represented using regular expressions. Regular languages do not require additional memory and are widely used in lexical analyzers and pattern matching.

5. Define a deterministic finite automaton (DFA).
Answer:
A DFA is a 5-tuple (Q, Σ, δ, q_0, F), where Q is a finite set of states, Σ is the input alphabet, δ is the transition function, q_0 is the start state, and F is the set of final states. Each state has exactly one transition for every input symbol.

6. How does an NFA differ from a DFA?
Answer:
In a DFA, for every state and input symbol, exactly one transition is defined. In an NFA, a state may have zero, one, or multiple transitions for an input symbol and may include ε-transitions. Despite this, both accept the same class of languages.

7. What is the importance of ε-transition in NFA?
Answer:
ε-transitions allow an NFA to move from one state to another **without consuming any input symbol**. They simplify automaton construction, especially when building NFAs from regular expressions. ε-transitions increase design flexibility without increasing computational power.

8. Describe the role of transition functions.
Answer:
The transition function defines how an FSM moves from one state to another based on the current state and input symbol. It controls the behavior of the automaton and determines whether a given input string will lead to an accepting or rejecting state.

9. Write any two applications of FSM.
Answer: Two applications of FSM are:
1. **Compiler design**, especially in lexical analysis for token recognition.
2. **Control systems** such as elevators, washing machines, and traffic signal controllers.

10. What is a state in FSM?
Answer:
A **state** in an FSM represents the current condition of the system after processing a sequence of inputs. It summarizes the past input history in a limited manner and determines the response to future input symbols.

11. Can FSM accept palindrome strings? Justify.
Answer:
No, a general FSM cannot accept all palindrome strings. Palindromes require comparing symbols from the beginning and end, which needs unlimited memory. FSMs have finite memory, so palindromes are not regular languages and cannot be fully recognized by FSMs.

12. What is the output of a finite state transducer?
Answer:
The output of a finite state transducer is a **sequence of output symbols** generated during transitions or states while processing an input string. The output depends on the structure of the transducer and is used in translation and signal processing.

13. Define canonical form in FSM.
Answer:
Canonical form of an FSM is its **minimal and unique representation** obtained after removing unreachable states and merging equivalent states. Two FSMs in canonical form are identical if they recognize the same regular language.

14. What do you understand by FSM minimization?
Answer: FSM minimization is the process of reducing the number of states in a finite automaton without changing the language it accepts. It involves removing unreachable states and merging equivalent states to obtain an efficient and simplified machine.

15. What is a Mealy machine?
Answer: A **Mealy machine** is a type of FSM in which the output depends on both the current state and the current input symbol. Outputs are associated with transitions, making Mealy machines generally faster and more compact than Moore machines.

LONG - 1

1. Explain the importance of studying the theory of computation.
Answer: The theory of computation is a fundamental area of computer science that studies what problems can be solved using computers and how efficiently they can be solved. It provides a formal framework for understanding different computational models such as finite automata, pushdown automata, and Turing machines.
This theory helps in identifying the limitations of computers by classifying problems into solvable, unsolvable, decidable, and undecidable categories. It forms the theoretical foundation for compiler design, programming languages, artificial intelligence, and algorithm development. By studying computational complexity, it also helps compare algorithms based on time and space requirements. Overall, the theory of computation guides the design of efficient systems and improves problem-solving skills in computer science.

2. Describe with a diagram the working of DFA and NFA.
Answer: A **Deterministic Finite Automaton (DFA)** processes input symbols by moving through exactly one state for each symbol. For every state and input, there is a unique next state. The computation follows a single path, and acceptance depends on whether the final state reached is an accepting state.
Example DFA diagram:
(q0) --a--> (q1)
(q1) --b--> (q2)
(q2) --a,b--> (q2)
A **Non-Deterministic Finite Automaton (NFA)** may have multiple possible transitions for a given input symbol or none at all. It can also include ε-transitions that do not consume input. The NFA accepts a string if **any path** leads to an accepting state.
Example NFA diagram:
(q0) --ε--> (q1)
(q0) --a--> (q0)
(q1) --b--> (q2)
Though their working differs, DFAs and NFAs recognize the same class of languages: regular languages.

3. Construct a DFA for strings ending with "ab".
Answer: To construct a DFA that accepts all strings ending with "ab", we track the recent input symbols.
States:
- q0: Start state (no match yet)
- q1: Last symbol read was 'a'
- q2: Last symbols were "ab" (accepting state)

Transition Table:

State	a	b
q0	q1	q0
q1	q1	q2
q2	q1	q0

Explanation:
The DFA reaches q2 only when the string ends with "ab". If additional symbols appear, it transitions accordingly. q2 is the only accepting state, ensuring that accepted strings must end with "ab".

4. Explain the process of FSM minimization with an example.
Answer:
FSM minimization is the process of reducing the number of states in a finite state machine without changing the language it accepts. This improves efficiency and simplifies implementation.
Steps of FSM minimization:
1. Remove all unreachable states.
2. Partition states into accepting and non-accepting groups.
3. Check equivalence of states by comparing transitions for each input symbol.
4. Merge equivalent states.

Example:
If two non-final states q1 and q2 transition to the same states for all inputs and neither is accepting, they are equivalent. These states can be merged into a single state. The resulting FSM is minimal and uniquely represents the language.

5. Write a detailed note on regular languages with examples.
Answer:
Regular languages are the simplest class of formal languages in the Chomsky hierarchy. They are recognized by finite automata such as DFAs and NFAs and can also be expressed using regular expressions. Regular languages do not require memory beyond a finite number of states.
They are closed under operations like union, intersection, concatenation, and Kleene star. Regular languages are widely used in practical applications such as lexical analysis in compilers, text searching, and pattern matching.
Examples of regular languages:
- All strings over {a, b} that end with "ab"
- Strings containing an even number of 0s
- Language described by regular expression (a+b)*

Due to their simplicity and efficiency, regular languages play a crucial role in computer science and software engineering.

UNIT 2

1. Define a Regular Expression.
Answer: A **Regular Expression (RE)** is a formal notation used to describe regular languages. It is constructed using symbols from an alphabet combined with operations such as union (+), concatenation, and Kleene star (*). Regular expressions provide a compact and mathematical way to specify patterns recognized by finite automata.

2. State Kleene's theorem.
Answer: Kleene's theorem states that a language is regular if and only if it can be described by a regular expression. It establishes the equivalence between regular expressions and finite automata, proving that both define exactly the same class of languages.

3. List two applications of Regular Expressions.
Answer: Two applications of regular expressions are:
1. **Lexical analysis** in compilers for token identification.
2. **Pattern matching and text searching** in tools such as editors, search engines, and scripting languages.

4. What is meant by simplifying a Regular Expression?
Answer: Simplifying a regular expression means rewriting it into a shorter or more efficient form without changing the language it represents. This is done using algebraic laws of regular expressions, making expressions easier to understand and implement.

5. Define a Regular Grammar.
Answer: A **Regular Grammar** is a type of formal grammar in which all productions are either right-linear or left-linear. Typical productions are of the form A → aB or A → a, where A and B are non-terminals and a is a terminal symbol.

6. How is a Regular Grammar related to a Regular Language?
Answer: A regular grammar generates a regular language. For every regular grammar, there exists a finite automaton that recognizes the same language, and for every regular language, a corresponding regular grammar can be constructed.

7. What is a Regular Language?
Answer: A regular language is a language that can be accepted by a finite automaton or expressed using a regular expression. Regular languages have limited memory requirements and are commonly used in lexical analysis and pattern processing.

8. Give an example of a non-regular language.
Answer: An example of a non-regular language is { $a^n b^n$ | n ≥ 1 }. This language requires equal numbers of a's and b's, which cannot be enforced by a finite automaton due to its limited memory.

9. What is the role of the Kleene Star in Regular Expressions?
Answer: The **Kleene Star (*)** allows repetition of a symbol or expression zero or more times. It enables the representation of infinite languages using finite notation and is essential for describing patterns with repetition.

10. Explain the union operation in Regular Expressions.
Answer: Union in regular expressions is represented by the symbol (+ or |). It denotes a choice between expressions. For example, a + b represents the language containing either 'a' or 'b'.

11. What is concatenation in the context of Regular Expressions?
Answer: Concatenation refers to placing expressions next to each other to form strings sequentially. For example, ab represents strings where symbol 'a' is followed immediately by symbol 'b'.

12. How can you prove a language is regular?
Answer:
A language can be proven regular by constructing a DFA or NFA that accepts it, writing a regular expression for it, or showing that it can be derived from known regular languages using closure properties.

13. Name one closure property of Regular Languages.
Answer: Regular languages are closed under **union**. This means that if L_1 and L_2 are regular languages, then their union $L_1 \cup L_2$ is also a regular language.

14. What is the significance of the empty string (ε) in Regular Expressions?
Answer: The empty string ε represents a string of length zero. It is important because it allows expressions to accept no input and plays a key role in defining Kleene star and ε-transitions in automata.

15. How does Arden's theorem help in manipulating Regular Expressions?
Answer: Arden's theorem provides a method to solve linear equations involving regular expressions. It is used to derive regular expressions from finite automata transition equations and helps simplify complex expressions.

LONG - 2

1. Explain Kleene's theorem in detail and prove that a language is regular if and only if it can be described by a Regular Expression.
Answer: Kleene's Theorem is a fundamental result in automata theory. It states that **a language is regular if and only if it can be described by a regular expression**. This theorem establishes the equivalence between **finite automata** and **regular expressions**.
Proof (Outline):
Kleene's theorem is proved in two parts:
(i) If a language is described by a regular expression, then it is regular:
For every basic regular expression (∅, ε, a), we can construct a finite automaton. Since regular expressions are closed under union, concatenation, and Kleene star, and finite automata are also closed under these operations, any language described by a regular expression can be accepted by a finite automaton.
(ii) If a language is regular, then it can be described by a regular expression:
If a language is accepted by a finite automaton, we can convert the automaton into a system of regular expression equations and solve them using **Arden's theorem** to obtain a regular expression.
Thus, regular expressions and finite automata describe exactly the same set of languages.

2. Discuss the applications of Regular Expressions in text processing and pattern matching.
Answer: Regular expressions are widely used in **text processing and pattern matching** due to their expressive and compact syntax. They allow users to describe complex string patterns efficiently.
In text processing, regular expressions are used for searching, filtering, and replacing text in editors such as **grep, sed, and awk**, and in programming languages like Python, Java, and Perl. They help identify patterns such as email addresses, phone numbers, and dates.
In pattern matching, regular expressions are used in **lexical analyzers** of compilers to identify tokens like keywords, identifiers, and literals. They are also essential in input validation, syntax highlighting, data extraction, and information retrieval systems. Their efficiency and flexibility make them a vital tool in software development.

3. Describe the process of simplifying a Regular Expression with an example.
Answer:
Simplifying a regular expression means transforming it into a simpler or shorter equivalent form without changing the language it represents. This is done using **algebraic laws of regular expressions**.
Common simplification rules:
- $r + r = r$
- $r\varepsilon = r$
- $r\emptyset = \emptyset$
- $r^* r^* = r^*$
- $(r^*)^* = r^*$

Example:
Given regular expression:
(a+a)b
Using the rule **a + a = a**, it simplifies to:
ab
Simplification helps reduce complexity, improves readability, and makes implementation in finite automata more efficient.

4. Explain the relationship between Regular Grammars and Regular Languages with examples.
Answer:
A **regular grammar** is a grammar that generates a **regular language**. Regular grammars can be **right-linear** or **left-linear**, where productions are of the form A → aB or A → a.
Relationship:
- Every regular grammar generates a regular language.
- For every regular language, there exists a regular grammar and a finite automaton that generates and accepts it respectively.

Example:
Grammar:
S → aS | b
This grammar generates the language:
$\{a^n b^n \mid n \geq 0\}$
This language is regular and can also be accepted by a DFA. Hence, regular grammars, regular languages, regular expressions, and finite automata are all equivalent in expressive power.

5. Prove that the language $\{a^n b^n \mid n \geq 0\}$ is not regular using the Pumping Lemma.
Answer:
To prove that
$L=\{a^n b^n \mid n \geq 0\}$
is **not regular**, we use the **Pumping Lemma for regular languages**.
Assumption:
Assume L is regular. Then, there exists a pumping length p such that any string $s \in L$ with $|s| \geq p$ can be split as s = xyz, satisfying:
1. $|xy| \leq p$
2. $|y| \geq 1$
3. $xy^iz \in L$ for all $i \geq 0$

Choose:
$s=a^p b^p$
Since $|xy| \leq p$, both x and y contain only a's. Let $y = a^k$ where $k \geq 1$.
Pump down (i = 0):
$xz= a^{p-k}b^p$
Now the number of a's ≠ number of b's, so $xz \notin L$.
This contradicts the Pumping Lemma assumption. Hence, the language $\{a^n b^n \mid n \geq 0\}$ **is not regular**.

UNIT 3

1. Define a grammar in Formal Language Theory.
Answer:
A grammar is a formal system used to generate a language. It is defined as a 4-tuple
G = (V, Σ, P, S), where V is a finite set of variables (non-terminals), Σ is a set of terminal symbols, P is a set of production rules, and S is the start symbol.

2. What is a derivation in the context of a grammar?
Answer: A derivation is the sequence of steps in which production rules are applied to the start symbol to generate a string in the language. It shows how a string is produced using grammar rules, either by leftmost or rightmost derivation.

3. What is a parse tree?
Answer: A parse tree is a tree representation of the syntactic structure of a string derived from a grammar. The root represents the start symbol, internal nodes represent non-terminals, and leaf nodes represent terminals or ε.

4. Explain what is meant by an ambiguous grammar.
Answer:
A grammar is ambiguous if there exists at least one string in its language that can be generated in more than one way, resulting in multiple distinct parse trees or multiple leftmost or rightmost derivations.

5. What is Chomsky Normal Form (CNF)?
Answer:
Chomsky Normal Form is a standard form of context-free grammar where every production is of the form A → BC or A → a, where A, B, C are non-terminals and a is a terminal symbol. It simplifies parsing algorithms.

6. Define a Pushdown Automaton (PDA).
Answer: A Pushdown Automaton is a computational model that extends a finite automaton by adding a stack. It is defined as a 7-tuple and uses the stack to store extra information, allowing it to recognize context-free languages.

7. What is the difference between a Deterministic and Non-deterministic PDA?
Answer: A Deterministic PDA (DPDA) has at most one possible move for a given input and stack symbol, while a Non-deterministic PDA (NPDA) may have multiple possible moves. NPDA is more powerful and can accept all context-free languages.

8. What type of languages can a PDA accept?
Answer: A Pushdown Automaton accepts **context-free languages (CFLs)**. These languages cannot be accepted by finite automata because they require additional memory, which the stack of the PDA provides.

9. What is the role of the stack in a PDA?
Answer:
The stack in a PDA provides memory beyond finite states. It is used to store symbols temporarily, helping the PDA handle recursive structures and matching patterns, such as balanced parentheses or equal numbers of symbols.

10. What is Greibach Normal Form (GNF)?
Answer: Greibach Normal Form is a type of context-free grammar where every production is of the form A → aα, where a is a terminal and α is a (possibly empty) string of non-terminals. It ensures derivations start with terminals.

11. How can you identify if a grammar is ambiguous?
Answer: A grammar is ambiguous if at least one string in its language has more than one parse tree or more than one leftmost or rightmost derivation. Finding such a string confirms ambiguity in the grammar.

12. What is the significance of a parse tree in grammar?
Answer: A parse tree shows the hierarchical structure of a string generated by a grammar. It helps in understanding the syntax, detecting ambiguity, and is widely used in compiler design during syntax analysis.

13. How does a Non-deterministic PDA differ from a finite automaton?
Answer: A Non-deterministic PDA differs from a finite automaton because it has a stack. This additional memory enables it to recognize context-free languages, whereas finite automata can recognize only regular languages.
14. What is meant by the halting problem for a PDA?
Answer: The halting problem for a PDA refers to determining whether the PDA will stop execution on a given input. Due to non-determinism and stack operations, it may be difficult to decide whether all computation paths halt.
15. Give an example of a context-free language accepted by a PDA.
Answer: An example of a context-free language is
$L = \{ a^n b^n \mid n \geq 0 \}$.
A PDA accepts this language by pushing symbols for each 'a' onto the stack and popping one symbol for each 'b'.

LONG - 3

1. Explain the concept of a grammar and its components with an example.
Answer: In formal language theory, a **grammar** is a mathematical model used to define the structure of a language. It specifies how strings in the language are generated. A grammar is formally defined as a 4-tuple:
G = (V, Σ, P, S)
Where:
- **V** is a finite set of variables (non-terminal symbols)
- **Σ** is a finite set of terminal symbols
- **P** is a finite set of production rules
- **S** is the start symbol

Example:
Let G = ({S}, {a, b}, P, S)
with production rules:
S → aSb | ε
This grammar generates the language of balanced strings where the number of a's equals the number of b's, such as ε, ab, aabb, aaabbb. Grammars are widely used in compiler design and language processing.

2. Discuss the process of constructing a parse tree for a given string in a context-free grammar.
Answer: A **parse tree** visually represents how a string is derived from a grammar. It shows the hierarchical structure of a sentence according to grammatical rules.
Steps to construct a parse tree:
1. Start with the **start symbol** as the root of the tree.
2. Apply production rules to expand non-terminals.
3. Each application of a rule creates child nodes in the tree.
4. Continue expanding until all leaves are terminal symbols or ε.
5. Read the leaves from left to right to obtain the derived string.

Example:
For grammar S → aSb | ε and string "aabb", productions are applied step by step. The parse tree reflects how each a is matched with a corresponding b, ensuring grammatical correctness.

3. Explain the concept of ambiguity in grammars and how it can be resolved with an example.
Answer: A grammar is **ambiguous** if there exists at least one string that has more than one parse tree or more than one leftmost or rightmost derivation. Ambiguity causes multiple interpretations of a single string, which is undesirable in programming languages.
Example:
Grammar:
E → E + E | E * E | id
Expression: id + id * id
This expression can have multiple parse trees, showing ambiguity.
Resolution:
Ambiguity can be resolved by restructuring the grammar to enforce precedence and associativity rules, such as separating expressions into terms and factors. This ensures each string has a unique parse tree.

4. Describe the steps to convert a context-free grammar to Chomsky Normal Form.
Answer: Chomsky Normal Form (CNF) is a standardized form of CFG useful for parsing algorithms.
Steps to convert CFG to CNF:
1. Remove **ε-productions** (except possibly from the start symbol).
2. Remove **unit productions** (A → B).
3. Remove **useless symbols** (non-generating or unreachable symbols).
4. Convert productions to the form A → BC or A → a.
5. Introduce new non-terminals where necessary to ensure the required format.

After these steps, the grammar is in CNF, making it suitable for algorithms like CYK parsing.

5. Explain the working of a Pushdown Automaton with a detailed example.
Answer:
A **Pushdown Automaton (PDA)** is an extension of a finite automaton equipped with a stack to provide additional memory. It is used to recognize context-free languages.
Working:
- The PDA reads an input symbol.
- Based on the current state, input symbol, and top of the stack, it performs an action: push, pop, or replace stack symbols.
- Transitions may occur with or without consuming input (ε-moves).
- The string is accepted if the PDA reaches a final state or the stack becomes empty.

Example:
Consider the language $L = \{ a^n b^n \mid n \geq 0 \}$.
The PDA pushes one symbol onto the stack for each 'a' read. When 'b' symbols are encountered, it pops one symbol per 'b'. If the stack becomes empty exactly when the input ends, the string is accepted.
This demonstrates how the stack allows PDAs to handle patterns that finite automata cannot.

UNIT - 4

1. What is a Context-Free Language (CFL)?
Answer: A Context-Free Language is a language generated by a context-free grammar (CFG). In a CFG, each production rule has a single non-terminal on the left-hand side. CFLs are used to describe nested structures such as parentheses, arithmetic expressions, and programming language syntax, and are recognized by pushdown automata.

2. Where do CFLs fit in the Chomsky hierarchy?
Answer: In the Chomsky hierarchy, CFLs are classified as **Type-2 languages**. They lie above regular languages (Type-3) and below context-sensitive languages (Type-1). CFLs are more powerful than regular languages but less powerful than context-sensitive languages.

3. How can you show that a language is context-free?
Answer: A language can be shown to be context-free by constructing a context-free grammar that generates it or by designing a pushdown automaton that accepts it. Since CFGs and PDAs are equivalent in power, either method is sufficient proof.

4. State the Pumping Lemma for Context-Free Languages.
Answer: The pumping lemma for CFLs states that for any context-free language L, there exists a pumping length p such that any string s in L with $|s| \geq p$ can be written as s = uvwxy, satisfying: $|vx| \geq 1$, $|vwx| \leq p$, and $uv^iwx^iy \in L$ for all $i \geq 0$.

5. What is a closure property of CFLs?
Answer: Context-free languages are closed under **union**. This means if L_1 and L_2 are CFLs, then their union $L_1 \cup L_2$ is also a CFL. A combined CFG or PDA can be constructed to recognize the union.

6. What is a Deterministic Context-Free Language (DCFL)?
Answer: A Deterministic Context-Free Language is a subset of CFLs that can be accepted by a Deterministic Pushdown Automaton (DPDA). Not all context-free languages are deterministic, making DCFLs less powerful than general CFLs.

7. How does a Deterministic PDA differ from a Non-deterministic PDA?
Answer: A Deterministic PDA allows at most one valid transition for a given state, input symbol, and stack symbol, whereas a Non-deterministic PDA may allow multiple transitions. NPDAs can recognize all CFLs, while DPDAs recognize only DCFLs.

8. Name a decidable question for CFLs.
Answer: The **membership problem** for CFLs is decidable. Given a context-free grammar and a string, it can be determined whether the string belongs to the language generated by the grammar using parsing algorithms.

9. What is an undecidable question for CFLs?
Answer: The **equivalence problem** for CFLs is undecidable. It is not possible to design an algorithm that determines whether two given context-free grammars generate the same language.

10. Define a Turing Machine.
Answer: A Turing Machine is a mathematical model of computation consisting of an infinite tape, a tape head, a finite set of states, and a transition function. It is more powerful than finite automata and PDAs and can simulate any algorithm.

11. How does a Turing Machine accept a language?
Answer: A Turing Machine accepts a language by entering an accepting state after reading an input string placed on the tape. If the machine halts in an accepting state, the string is accepted; otherwise, it is rejected or the machine may loop.

12. What is the role of the tape in a Turing Machine?
Answer: The tape serves as both input storage and unlimited memory for the Turing Machine. It allows reading, writing, and moving left or right, enabling complex computations that require unbounded memory.

13. Give an example of a Context-Free Language.
Answer: An example of a context-free language is $L = \{ a^n b^n \mid n \geq 0 \}$. This language can be generated by the grammar S → aSb | ε and accepted by a pushdown automaton.

14. Why is the language $\{a^n b^n c^n \mid n \geq 0\}$ not context-free?
Answer: The language $\{a^n b^n c^n \mid n \geq 0\}$ is not context-free because it requires matching three different symbols with equal counts. A PDA has only one stack and cannot keep track of two independent counts, which can be proven using the pumping lemma for CFLs.

15. What is closure under union for CFLs?
Answer:
Closure under union means that if two languages L_1 and L_2 are context-free, then their union $L_1 \cup L_2$ is also context-free. A new CFG can be created by introducing a new start symbol that derives the start symbols of both grammars.

LONG – 4

1. Explain the position of Context-Free Languages in the Chomsky hierarchy with examples.

Answer: The Chomsky hierarchy classifies formal languages into four types based on their expressive power. Context-Free Languages (CFLs) belong to **Type-2** in this hierarchy. They are more powerful than **Type-3 Regular Languages** but less powerful than **Type-1 Context-Sensitive Languages** and **Type-0 Recursively Enumerable Languages**.

CFLs are generated by **Context-Free Grammars (CFGs)**, where each production rule has a single non-terminal on the left-hand side. They are recognized by **Pushdown Automata (PDA)**, which use a stack for memory.

Example:

The language L = { $a^n b^n$ | n ≥ 0 } is a CFL but not a regular language.

A regular language example below CFLs is L = (a + b)*.

A language above CFLs is { $a^n b^n c^n$ | n ≥ 0 }, which is context-sensitive.

Thus, CFLs form an important middle layer in the hierarchy, capable of representing nested and recursive structures.

2. Discuss how to prove a language is context-free by constructing a PDA with an example.

Answer: A language can be proved context-free by constructing a **Pushdown Automaton (PDA)** that accepts it. Since PDAs and CFGs are equivalent in computational power, acceptance by a PDA guarantees the language is context-free.

A PDA uses a **stack** to store symbols, allowing it to handle nested dependencies that finite automata cannot.

Example:

Consider the language L = { $a^n b^n$ | n ≥ 0 }.

Construction idea:

- Push one symbol onto the stack for each input 'a'.
- For each input 'b', pop one symbol from the stack.
- Accept the string if the stack is empty at the end of input.

This PDA ensures equal numbers of a's and b's. Since a PDA exists for this language, it is context-free. Hence, constructing a PDA is a valid method to prove a language is context-free.

3. Explain the Pumping Lemma for Context-Free Languages and its application with a detailed example.

Answer: The Pumping Lemma for Context-Free Languages provides a necessary condition for a language to be context-free. It is mainly used to **prove that a language is not context-free**.

The lemma states that for any CFL L, there exists a pumping length p such that any string s ∈ L with |s| ≥ p can be written as:

s = uvwxy

such that:

1. |vx| ≥ 1
2. |vwx| ≤ p
3. $uv^i wx^i y$ ∈ L for all i ≥ 0

Application Example:

Consider L = { $a^n b^n c^n$ | n ≥ 0 }.

Assume L is context-free. Choose s = $a^p b^p c^p$.

According to the lemma, s can be split as uvwxy. However, v and x can affect at most two types of symbols. Pumping v and x changes the balance among a, b, and c, resulting in strings not in L.

This contradiction proves L is **not context-free**.

4. Discuss the closure properties of Context-Free Languages with examples for union, concatenation, and Kleene star.

Answer: Closure properties describe operations under which a class of languages remains within the same class. CFLs are closed under **union, concatenation, and Kleene star**.

Union:

If L_1 and L_2 are CFLs, then $L_1 \cup L_2$ is also a CFL.

Example:

L_1 = { $a^n b^n$ }, L_2 = { $a^n c^n$ }

Their union is context-free.

Concatenation:

If L_1 and L_2 are CFLs, then $L_1 L_2$ is also a CFL.

Example:

L_1 = { $a^n b^n$ }, L_2 = { c^m }

Then L = { $a^n b^n c^m$ } is context-free.

Kleene Star:

If L is a CFL, then L* is also a CFL.

Example:

If L = { $a^n b^n$ }, then repeating strings of this form any number of times still forms a CFL.

Thus, CFLs support many useful language-building operations.

5. Explain the significance of Deterministic Context-Free Languages and their differences from general CFLs.

Answer: Deterministic Context-Free Languages (DCFLs) are a proper subset of CFLs that can be accepted by **Deterministic Pushdown Automata (DPDA)**. Their significance lies in efficient parsing and practical implementation in compilers.

Differences from CFLs:

- All DCFLs are CFLs, but not all CFLs are DCFLs.
- A DPDA allows only one possible move for a given input and stack symbol, while an NPDA allows multiple choices.
- DCFLs are closed under complement, whereas CFLs are not.

Example:

L = { $a^n b^n$ } is a DCFL.

L = { ww^r | w ∈ {a, b}* } is CFL but not deterministic.

DCFLs are important because programming language syntax is designed to be deterministic for efficient parsing.

UNIT - 5

1. What is a multi-tape Turing Machine?

Answer: A multi-tape Turing Machine is an extended model of a standard Turing Machine that uses **more than one tape**, each with its own read/write head. All tapes operate simultaneously during computation. Although it is more efficient than a single-tape TM, it does **not increase computational power**. Any multi-tape TM can be simulated by a single-tape TM. Multi-tape TMs improve algorithmic efficiency by separating input, work area, and output onto different tapes.

2. Define a Linear Bounded Automaton (LBA).

Answer: A Linear Bounded Automaton is a restricted form of a Turing Machine whose tape is limited to a length that is **linearly bounded by the size of the input**. An LBA does not have infinite tape and works within a fixed memory boundary. LBAs recognize exactly the class of **Context-Sensitive Languages (CSLs)**. They are more powerful than PDAs but less powerful than full Turing Machines.

3. What is an algorithm in the context of computation?

Answer: An algorithm is a **finite, well-defined sequence of instructions** used to solve a problem or perform a computation. It must always produce a correct output for valid inputs and terminate after a finite number of steps. In computation theory, algorithms are often modeled using Turing Machines, which formalize the steps of computation.

4. What does it mean for a language to be decidable?

Answer: A language is decidable if there exists a Turing Machine that **always halts** and correctly decides whether a string belongs to the language. For any input string, the TM accepts if the string is in the language and rejects otherwise. Decidable languages correspond to problems that can be solved algorithmically with guaranteed termination.

5. State the Halting Problem for Turing Machines.

Answer: The Halting Problem asks whether a given Turing Machine **halts** on a given input string. Formally:

Given a TM M and an input w, will M halt on w?

Alan Turing proved that this problem is **undecidable**, meaning no general algorithm can determine halting behavior for all possible Turing Machines.

6. What is the Post Correspondence Problem (PCP)?

Answer: The Post Correspondence Problem is a decision problem involving **matching strings**. Given two lists of strings over the same alphabet, the question is whether there exists a sequence of indices such that the concatenation of strings from both lists becomes equal. PCP is **undecidable** and is used in computability theory to prove other problems undecidable.

7. Define the complexity class P.

Answer: P is the complexity class consisting of decision problems that can be solved by a deterministic Turing Machine in **polynomial time**. These problems are considered efficiently solvable. Examples include sorting, searching, and graph reachability. P helps classify problems based on computational resources.

8. Define the complexity class NP.

Answer:

NP is the class of decision problems for which a solution, once given, can be **verified in polynomial time** by a deterministic Turing Machine. NP problems may not necessarily be solved quickly, but their solutions can be checked quickly. Examples include SAT and Traveling Salesman decision versions.

9. What is a qubit in quantum computation?

Answer:

A qubit (quantum bit) is the basic unit of quantum information. Unlike a classical bit that is either 0 or 1, a qubit can exist in a **superposition** of both states simultaneously. Qubits also exhibit **entanglement**, enabling quantum computers to perform computations that classical computers cannot efficiently achieve.

10. State the Church–Turing thesis.

Answer:

The Church–Turing thesis states that any function that can be effectively computed by an algorithm can be computed by a **Turing Machine**. It establishes the TM as the standard model of computation. Although not a formal theorem, it is widely accepted based on strong evidence and equivalence of computational models.

11. What is a non-deterministic Turing Machine?

Answer:

A Non-deterministic Turing Machine (NTM) is a theoretical TM model where **multiple moves are possible** from a single configuration. It explores many computational paths simultaneously. An NTM accepts a string if **at least one** computation path leads to an accepting state. NTMs help define complexity classes like NP.

12. What type of languages can an LBA accept?

Answer:

An LBA accepts exactly the class of **Context-Sensitive Languages (CSLs)**. These languages require memory that grows proportionally to input size. LBAs are more powerful than PDAs (which accept CFLs) but restricted compared to general Turing Machines. Examples include the language {$a^n b^n c^n$ | n ≥ 0}.

13. Give an example of a decidable language.

Answer:

An example is the language **L = {$0^n 1^n$ | n ≥ 0}**, which is decidable by a TM that compares counts of 0s and 1s. The TM halts for all inputs, accepting strings with equal numbers of 0s and 1s and rejecting others. All regular and context-free languages are also decidable.

14. Why is the Halting Problem undecidable?

Answer:

The Halting Problem is undecidable because there is **no general algorithm** that can determine, for every TM and input, whether the machine will halt. Turing proved this by contradiction using a diagonalization argument. If a universal halting-decider existed, it would lead to logical inconsistency.

15. What is the significance of the complexity class P?

Answer:

The class P represents decision problems solvable **efficiently** on deterministic machines. It defines the boundary between feasible and infeasible computation. Understanding P helps classify real-world problems and is central to the famous **P vs NP** question. If P = NP, many currently hard problems would become efficiently solvable.

LONG - 5

1. Explain the concept of a multi-tape Turing Machine and how it differs from a single-tape TM with an example.

Answer: A **multi-tape Turing Machine (MTTM)** is an extension of the classical single-tape Turing Machine. While a standard TM uses **one tape** and **one read–write head**, a multi-tape TM uses **multiple tapes**, each equipped with its own read–write head. In each computation step, the machine reads symbols from all tapes, changes states, writes new symbols, and moves each head independently.

Differences from a Single-Tape TM

1. **Number of tapes:**
   Single-tape TM → 1 tape;
   Multi-tape TM → 2 or more tapes.
2. **Speed and efficiency:**
   Multi-tape TMs can perform tasks faster because intermediate calculations can be stored on separate tapes.
   Example: copying a string takes **O(n)** time on multi-tape but **O(n^2)** on a single-tape machine.
3. **Parallel access:**
   Multiple heads allow simultaneous reading/writing of multiple regions of input.
4. **Computational power:**
   Both models recognize the same class of languages (recursively enumerable).
   Multi-tape TM **does not add power**, only efficiency.

Example

To **copy** a string w:

- **Tape 1:** Contains input w
- **Tape 2:** Initially blank
- **Steps:**
  - Read each symbol from Tape 1.
  - Write the same symbol to Tape 2.
  - Move right on both tapes.

Result: Tape 2 contains a copy of w.

A single-tape TM must repeatedly shift symbols to create space, making it slower.

Thus, MTTMs help improve efficiency while preserving computational equivalence to single-tape TMs.

2. Discuss the Linear Bounded Automaton model and its role in accepting context-sensitive languages.

Answer: A **Linear Bounded Automaton (LBA)** is a restricted version of a Turing Machine designed to recognize **Context-Sensitive Languages (CSLs)**.

Unlike a general TM with an infinite tape, an LBA uses a tape whose size is **linearly limited** by the length of the input.

Key Characteristics

1. **Space-limited tape:**
   The tape length is O(n), where n = size of input.
2. **No infinite movement:**
   The TM cannot move beyond the boundary markers.
3. **Equivalent to Context-Sensitive Grammars (Type-1 grammars).**

Role in Accepting CSLs

LBAs recognize exactly the class of **context-sensitive languages**, which require more computational power than CFLs.

Example of a CSL

The language:

L = { $a^n b^n c^n$ | n ≥ 0 }

is **not** context-free but is context-sensitive.

An LBA can process this language by:

- marking each group of a, b, and c,
- ensuring equal numbers,
- using limited tape space.

Significance

- LBAs bridge the gap between PDAs and full Turing Machines.
- They model computations requiring memory proportional to input size.
- They are important in studying space complexity, primarily the class **NSPACE(n)**.

Thus, LBAs are the formal model for **context-sensitive computation**.

3. Explain the concept of decidability and provide examples of decidable and undecidable languages.
Answer: A language is **decidable** if there exists a Turing Machine that **always halts** and for every input string:

- **Accepts** if the string belongs to the language
- **Rejects** if it does not

Decidable languages correspond to problems with a **complete algorithmic solution**.
Examples of Decidable Languages

1. **Regular languages**
   E.g., L = { w | w contains an equal number of 0's mod 2 }
   A DFA decides this.
2. **Context-free languages**
   E.g., L = { $a^n b^n$ | n ≥ 0 }
   A PDA or TM can always determine membership.
3. **Arithmetic checking language**
   L = { x | x is a valid arithmetic expression }
   A parsing algorithm halts for all inputs.

Undecidable Languages
A language is undecidable if **no TM can halt** on all inputs and correctly decide membership.
Examples:

1. **Halting Problem Language**
   L = { ⟨M, w⟩ | M halts on w }
   Proven undecidable by Turing.
2. **Post Correspondence Problem (PCP)**
   L = { instances of PCP that have a solution }
   Undecidable for general cases.
3. **Ambiguity Language for CFGs**
   L = { G | grammar G is ambiguous }
   No algorithm exists to decide ambiguity for all CFGs.

Conclusion
Decidability distinguishes solvable problems from inherently unsolvable ones, forming the foundation of computability theory.

4. Discuss the Halting Problem for Turing Machines and prove why it is undecidable.
Answer: The **Halting Problem** asks:
Given a Turing Machine M and input w, will M halt on w?
Alan Turing proved in 1936 that this problem is **undecidable**, meaning no algorithm can solve it for all possible inputs.

Statement of the Problem
Define the language:
H = { ⟨M, w⟩ | M halts on input w }
Claim: **H is undecidable.**

Proof (By Contradiction)
Assume there exists a TM **HaltCheck** that decides H:

- HaltCheck(M, w) = YES if M halts on w
- HaltCheck(M, w) = NO if M loops forever

Now construct a new TM D:
D(x):

1. Run HaltCheck(x, x)
2. If HaltCheck says "halts," then D **loops forever**
3. If HaltCheck says "loops," then D **halts**

So:

- If x halts on itself → D loops
- If x loops on itself → D halts

Now ask: **Does D halt on input D?**
Case 1: HaltCheck says D(D) halts → D loops → contradiction
Case 2: HaltCheck says D(D) loops → D halts → contradiction
Both cases contradict the assumption.
Thus, **HaltCheck cannot exist**, meaning:
The Halting Problem is Undecidable.

Importance

- Forms the basis of computability limits.
- Shows there exist tasks no computer can solve.
- Used to prove other problems undecidable (PCP, CFG ambiguity, equivalence).

5. Explain how a non-deterministic Turing Machine works and its equivalence to a deterministic TM.
Answer: A Non-deterministic Turing Machine (NTM) is a theoretical extension of a TM where multiple transitions are possible from a single configuration. It can be seen as exploring **multiple computation branches simultaneously**.
How an NTM Works

- In a given state and tape symbol, the transition function may allow **many** next moves.
- Computation forms a **tree** of configurations.
- If **any** branch leads to an accepting state, the NTM **accepts** the input.
- It rejects only if **all** branches reject.

Formal Description
δ(q, a) = { (p, b, D), ... }
This means multiple next states and actions are possible.
Example
An NTM deciding whether a string over {0,1} contains at least one '1':

- It non deterministically selects a position.
- If that position contains '1', accept.

This is equivalent to checking all positions in parallel.

Equivalence to Deterministic TM
Even though NTMs allow parallel exploration, they **do not have more power**.
Theorem:
Every NTM has an equivalent deterministic TM (DTM).
Proof Idea
A deterministic TM can simulate all computation branches of an NTM by:

- Enumerating all branches in a breadth-first manner
- Storing configurations on a separate work tape
- Exploring one step at a time

If any branch accepts, the DTM accepts.
Thus:
NTM languages = DTM languages = Recursively enumerable languages
Significance

- NTMs help define complexity class NP.
- NTMs simplify algorithm design.
- They are theoretical tools—not physically realizable machines.

1. Construct a DFA that accepts strings over {a,b} where every a is followed by exactly two b's.
Answer:
We must accept exactly those strings in which every occurrence of a is immediately succeeded by two b symbols ("abb" as a block). Any a that is not followed by two b's (because of end-of-string or a different symbol) must be rejected. b may appear independently anywhere.
Informal idea / states:

- q0 — start state; no pending requirement; **accepting** (also accepts empty string).
- q1 — we have just read a and now **expect the first b** (not accepting).
- q2 — we have read a then one b and now **expect the second b** (not accepting).
- q_dead — error / reject (once reached, stay there).

Transitions:

- From q0: on a → q1; on b → q0.
- From q1: on b → q2; on a → q_dead (because an a appeared instead of the required b).
- From q2: on b → q0 (completed the abb); on a → q_dead.
- From q_dead: on a or b → q_dead (loop).

Accepting states: { q0 } only — because q1 or q2 indicate an incomplete a→bb requirement.
Example checks:

- abb : q0 --a→ q1 --b→ q2 --b→ q0 — accepted.
- babbabb : accepted (every a followed by bb).
- ab or a or aba : rejected (incomplete bb after an a).

(You can draw the 4-state DFA with arrows as above. This DFA is minimal for this constraint.)

2. Construct a DFA that accepts strings over {a,b} where the third symbol of the string from the end is a.
Answer: We must accept strings whose symbol at position |w|-2 (counting last symbol as position |w|) is a. Strings of length less than 3 are **rejected**. A DFA can achieve this by **remembering the last three symbols seen** (sliding window). When the input ends, the window holds the final three symbols; accept if the *first* of those three is a.
State design (conceptual):

- Use states that encode the last up-to-3 symbols seen. Before the input reaches length 3 we keep partial-history states; after length ≥ 3 we store exactly the last 3 symbols.
- Let ⊥ denote "no symbol yet".

Define states:

- q0 = ⊥ (start, length 0)
- q1_a, q1_b = last 1 symbol is a or b (length 1)
- q2_aa, q2_ab, q2_ba, q2_bb = last 2 symbols (length 2)
- s_aaa, s_aab, s_aba, s_abb, s_baa, s_bab, s_bba, s_bbb = last 3 symbols (length ≥ 3)

Initial transitions (filling the window):

- q0 on a → q1_a; on b → q1_b.
- q1_a on a → q2_aa; on b → q2_ab.
- q1_b on a → q2_ba; on b → q2_bb.
- q2_xy on z → s_xyz (e.g., q2_ab on a → s_aba).

Stable transitions for 3-symbol states:

- From s_{x y z} on input d (d ∈ {a,b}) → s_{y z d} (slide window left and append new symbol).

Acceptance:

- Accepting states are the 3-symbol states whose first symbol is a:
  { s_aaa, s_aab, s_aba, s_abb }.
- All partial-history states (q0, q1_*, q2_*) are non-accepting because string length < 3.

Explanation / correctness:
At the end of input, the DFA will be in the state representing the final three symbols (if length ≥ 3). Acceptance checks whether the first of those three is a. This construction uses 1 + 2 + 4 + 8 = 15 states in total (compact and explicit). One can minimize some unreachable combinations depending on requirements, but the above is the direct, clear DFA.

3. Create a DFA over {a,b} that accepts all strings that do not start or end with b.
Answer: Interpretation: strings that **neither start with b nor end with b**. That is, the first symbol (if any) ≠ b and the last symbol (if any) ≠ b. The empty string ε has no first/last symbols, so it **does not start or end with b** — typically accepted (unless problem statement excludes it; here we accept ε).
State idea:
We must reject immediately if the first symbol is b. We must also remember the **most recent symbol** during scanning because acceptance at the end depends on whether the last symbol is b or not.
States:

- q0 — start state; length 0; **accepting** (represents empty string; last symbol = none)
- q1 — we have started and first symbol was a; the last symbol currently is a; **accepting**
- q2 — we have started and first symbol was a; the last symbol currently is b; **non-accepting** (because end currently b)
- q_dead — first symbol was b (rejected forever)

Transitions:

- From q0: on a → q1; on b → q_dead.
- From q1 (last seen a): on a → q1 (last a); on b → q2 (last b).
- From q2 (last seen b): on a → q1 (last a); on b → q2 (last b).
- From q_dead: on a or b → q_dead (loop).

Accepting states: { q0, q1 } (empty string and strings whose last symbol is a, provided the first was not b).
Examples:

- ε → accepted (q0).
- a → accepted (q1).
- abbaa → accepted (started with a, ends with a).
- b, ba, abb → rejected (either start with b or end with b).

This DFA uses 4 states and is minimal for the stated constraint.

4. Explain the Chomsky hierarchy of languages.
Answer: The **Chomsky hierarchy** classifies formal languages (and their grammars) into four types by increasing expressive power:

1. **Type-3 — Regular languages**
   - Generated by **regular grammars** (right-linear or left-linear).
   - Recognized by **finite automata** (DFA/NFA).
   - Closed under union, concatenation, Kleene star, complement, intersection (with regular), etc.
   - Example: (a+b)*, { a^n | n ≥ 0 }, { w | w ends with "ab" }.
2. **Type-2 — Context-free languages (CFLs)**
   - Generated by **context-free grammars** (productions A → α, where A is single nonterminal).
   - Recognized by **pushdown automata (PDA)**.
   - Useful for programming language syntax (nested structures).
   - Example: { $a^n b^n$ | n ≥ 0 }, arithmetic expressions.
3. **Type-1 — Context-sensitive languages (CSLs)**
   - Generated by **context-sensitive grammars** (productions of the form αAβ → αγβ with |γ| ≥ 1).
   - Recognized by **Linear Bounded Automata (LBA)** — TMs with tape bounded linearly by input size.
   - Example: { $a^n b^n c^n$ | n ≥ 0 } (context-sensitive but not context-free).
4. **Type-0 — Recursively enumerable languages (RE)**
   - Generated by unrestricted grammars (no restrictions on productions).
   - Recognized by **Turing machines** (may not halt on inputs not in the language).
   - This is the most general class of languages describable by algorithms.
   - Example: Any language decidable or semi-decidable by a TM.

Relationships:
Regular ⊂ Context-Free ⊂ Context-Sensitive ⊂ Recursively Enumerable. Each inclusion is strict. The hierarchy organizes languages by the computational resources (memory, stack, tape) required for recognition.

5. Define an alphabet, string, and language with examples.
Answer:
These are the basic building blocks of formal language theory:

- **Alphabet (Σ):**
  A finite, non-empty set of symbols.
  Example: Σ = {a, b} or Σ = {0,1}.
- **String (or word):**
  A finite sequence of symbols drawn from an alphabet. The length of a string w is denoted |w|. The empty string (no symbols) is denoted ε.
  Examples: Over Σ = {a,b}: ε, a, b, abba, aaab.
- **Language:**
  Any set of strings over a given alphabet — i.e., L ⊆ Σ* (where Σ* is the set of all finite strings over Σ). A language may be finite or infinite.
  Examples:
  - L1 = { a^n b^n | n ≥ 0 } (all strings with n a's followed by n b's).
  - L2 = { w ∈ {0,1}* | w has an even number of 0s } (regular).
  - L3 = ∅ (empty language) and { ε } (language with just the empty string).

Notes:

- Σ* denotes all strings over Σ (including ε).
- Σ+ denotes all non-empty strings (Σ* without ε).

These definitions are foundational for describing grammars, automata, and computational problems.